more than one condition to be true, the case analysis always executes only one case, the first one that is true. If the three conditions are disjoint, then the four cases are (1) C1, (2) C2, (3) C3, (4) (not C1) and (not C2) and (not C3).

Example 7.3. We want an algorithm that compares two numbers and produces the result as

$$\text{compare}(a, b) = \begin{cases} 1- & \text{if } a < b \\ 0 & \text{if } a = b \\ 1 & \text{if } a > b \end{cases}$$

We can split the state into an exhaustive set of 3 disjoint cases: a < b, a = b, and a> b. Then we can define compare() using a case analysis.

```
1.  compare(a, b)
2.      case a < b
3.          result := -1
4.      case a = b
5.          result := 0
6.      else -- a > b
7.          result : = 1
```

7.2.4 Iterative statement

An iterative process executes the same action repeatedly, subject to a condition C. If C is a condition and S is a statement, then

```
while C
  S
```

is a statement, called an iterative statement, that describes the following action:

1. Test whether C is true or false.
2. If C is true, then do S and go back to step 1; otherwise do nothing.

The iterative statement is commonly known as a loop. These two steps, testing C and executing S, are repeated until C becomes false. When C becomes false, the loop ends, and the control flows to the statement next to the iterative statement. The condition C and the statement S are called the loop condition and the loop body, respectively. Testing the loop condition and executing the loop body once is called an iteration. not C is known as the termination condition.

Iterative control flow is depicted in the flowchart of Figure 7.6. Condition C has two outgoing arrows, true and false. The true arrow points to S box. If C is true, S box is executed and control flows back to C box. The false arrow points to the box after the iterative statement (dotted box). If C is false, the loop ends and the control flows to the next box after the loop.

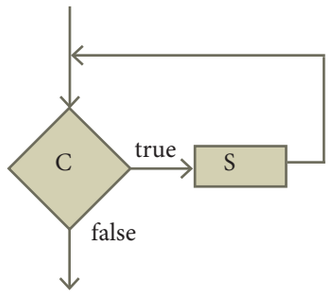


Figure 7.6: Iterative control flow

Example 7.4. Construct an iterative algorithm to compute the quotient and remainder after dividing an integer A by another integer B.

We formulated the specification of the algorithm in Example 6.6 as

```
divide (A , B)
-- inputs:     A is an integer and B ≠ 0
-- outputs :   q and r such that A = q X B
      + r and
--              0 ≤ r < B
```

Now we can construct an iterative algorithm that satisfies the specification.

Chapter 7 Page 088-101.indd 93 3/24/2020 9:14:50 AM